# OptiWave

Deenabandhan and Aditi V

## Introduction

Optimisation and it's necessity is something we are all well aware of. The ability to get better outcomes and accurate predictions, while using fewer resources is an intrinsic part of human nature and the guiding force in evolution. Through this project we aim to study mathematical models that bring real world problems on paper, express them in forms we are comfortable with and apply algorithms that help sieve this data and return an optimised version of what we started out with.

- In this project, we try to remove the noise in a given signal which is our end goal.
- So we will discuss various mathematical techniques of optimisation and try to improve upon the methods.
- We would like to familiarise you with two key optimisation techniques; which will not only be useful later in the project, but are just as useful on their own.

## Fourier Analysis

- Does the name sound familiar? Indeed, Fourier Analysis stands as one of the most profound and elegant achievements of the 19th century in mathematics and engineering.
- Its most captivating feature is the ability to transform a time-domain signal into its frequency-domain representation, unveiling the hidden spectrum of frequencies that make up the signal.
- You might already know how periodic signals can be expressed as a sum of simple sine and cosine waves—a concept rooted in Fourier series.
- But what about aperiodic signals? Fascinatingly, we can extend this idea by letting $T \to \infty$, giving rise to the Fourier Transform. The mathematical representation is as follows:

$$X(j\omega) = \int_{-\infty}^{\infty} x(t)e^{-j\omega t}\,dt$$

  And the corresponding inverse transform, which reconstructs the time-domain signal from its frequency components, is given by:

$$x(t) = \frac{1}{2\pi}\int_{-\infty}^{\infty} X(j\omega)e^{j\omega t}\,d\omega$$

- The frequency domain representation is comprehensively explained in the following image.

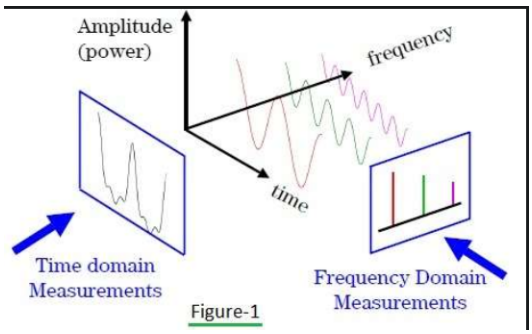



Figure-1

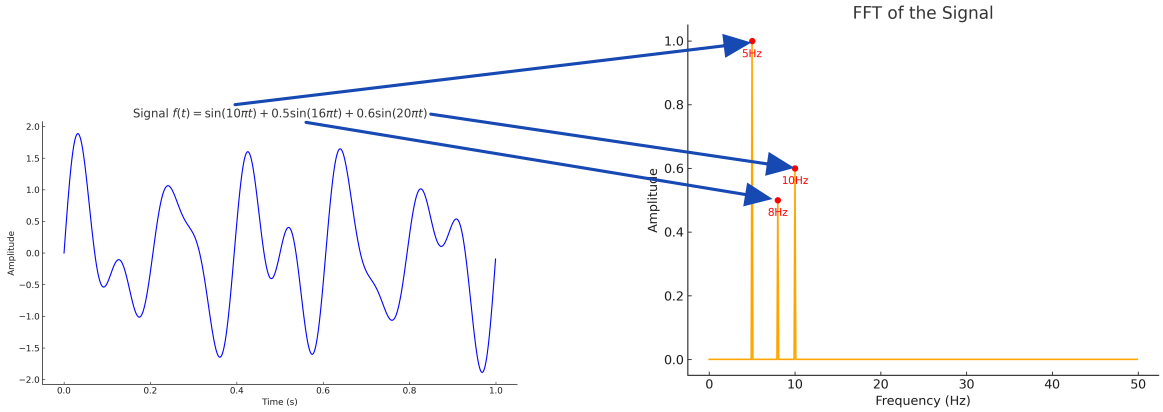
- So, why does this matter? One compelling application of Fourier analysis is noise reduction. By analyzing a signal in the frequency domain, we can isolate and mitigate unwanted components, improving the overall signal quality.

## De-noising Using Fourier Analysis

- As we've discussed, the Fourier Transform allows us to represent any time-domain signal in the frequency domain, breaking it down into its constituent frequencies. This is an invaluable tool for analyzing and processing signals.
- In the frequency spectrum, we can identify the dominant frequencies that define the signal. For instance, consider the signal:

$$f(t) = \sin(10\pi t) + 0.5\sin(16\pi t) + 0.6\sin(20\pi t)$$

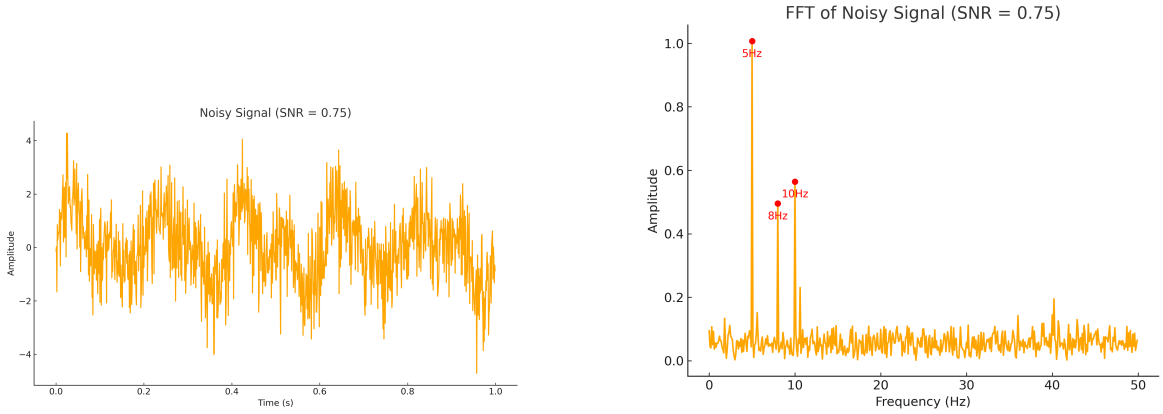
Its Fourier Transform, $X(j\omega)$, is shown in the figure 1:




(a) Time-domain signal (b) Frequency-domain signal

Signal without noise

- From the spectrum, we observe sharp peaks at specific frequencies, corresponding to the dominant sinusoidal components of the signal.
- Now, let's add some noise to the signal and analyze its effect using the Fourier Transform:




(a) Time-domain signal (b) Frequency-domain signal

Signal with noise

- In the noisy spectrum, we notice additional frequency components that do not belong to the original signal. These represent the noise.

- Using the Fourier Transform, we can effectively remove noise from the signal. There are two main approaches to achieve this:

  1. **Thresholding**: By setting a threshold value, we can filter out unwanted frequency components whose amplitudes fall below this threshold. You can refer to this for further reading.
  2. **Low-Pass Filtering** : Noise is often concentrated in higher frequencies, while the core signal lies in lower frequencies. By ignoring components below a certain minimum frequency, we can retain the essential parts of the signal. This technique is known as low-pass filtering.

- Once the noise is removed, we can reconstruct the clean signal using the inverse Fourier Transform, restoring its original form with improved quality.

Task 1: From the above concepts, try to denoise the following signal and find the frequencies present in it.

## SVD: Singular Value Decomposition

- **What is SVD?**
  SVD or singular value decomposition is a method of breaking down a matrix into the sum of numerous rank one matrices.
  If you are someone who wears spectacles, follow this train of thought. Think of the input of your sense of sight as one giant matrix, without a lens in front of your eyes, everything appears nebulous and blurry. The moment you add one more layer to your perception, the finer details become dominant and the world begins to make sense again. If you were to represent the world that is a matrix (*literally*) as a sum of matrices where each matrix, successively added, brings you closer to the actual matrix, how would you go about it??

Task 2: Use this (and other videos from the YouTube playlist) to get an intuitive understanding of SVD.

Find the SVD of $\begin{bmatrix} 3 & 2 & 2 \\ 2 & 3 & -1 \end{bmatrix}$

- **Optimzation meets SVD**
  **Image compression** is an excellent way to see how all of this plays out. Using SVD we can express the our matrix as the sum of multiple rank one matrices. However the questions that remain are: are all of these rank one matrices equally valuable? What parameter would help us decide which matrices we can afford to leave out? And most importantly how helpful would this kind of compression be on an image whose pixel values are generated randomly?

Task 3: On an image of your choice, perform image compression by following the steps given here. Use this newly found insight to answer the questions above. (The last question brings up some interesting perspective on what we might consider undesirable in data)

**Task 1:**

Program to denoise the given csv file (python):

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

file = "signal.csv" # Path of the file
data = pd.read_csv(file)

signal = data['NoisySignal'].values
time = data['Time'].values
sampling_rate = 1 / (time[1] - time[0])
n = len(signal)

fft_signal = np.fft.fft(signal)
freq = np.fft.fftfreq(n, d=1/sampling_rate)
magnitude = np.abs(fft_signal)

threshold = np.max(magnitude) * 0.5  # Keep frequencies with magnitude > 50% of max
# After experimenting different values 50% seemed the best even though it seems high
fft_signal_clean = fft_signal.copy()
fft_signal_clean[magnitude < threshold] = 0

denoised_signal = np.real(np.fft.ifft(fft_signal_clean))

plt.figure(figsize=(15, 10))

plt.subplot(4, 1, 1)
plt.plot(time, signal, 'b-', label='Original Signal', alpha=0.7)
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.title('Original Signal')
plt.grid(True)
plt.legend()

plt.subplot(4, 1, 2)
positive_freq_mask = freq >= 0
plt.plot(freq[positive_freq_mask], magnitude[positive_freq_mask], 'b-',
         label='Original Frequency Spectrum')
plt.axhline(y=threshold, color='r', linestyle='--', label='Threshold')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude')
plt.title('Original Frequency Spectrum')
plt.grid(True)
plt.legend()
```

```
plt.subplot(4, 1, 3)
magnitude_clean = np.abs(fft_signal_clean)
plt.plot(freq[positive_freq_mask], magnitude_clean[positive_freq_mask], 'r-',
         label='Denoised Frequency Spectrum')
plt.xlabel('Frequency (Hz)')
plt.ylabel('Magnitude')
plt.title('Denoised Frequency Spectrum')
plt.grid(True)
plt.legend()

plt.subplot(4, 1, 4)
plt.plot(time, denoised_signal, 'r-', label='Denoised Signal', linewidth=1.5)
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.title('Denoised Signal')
plt.grid(True)
plt.legend()

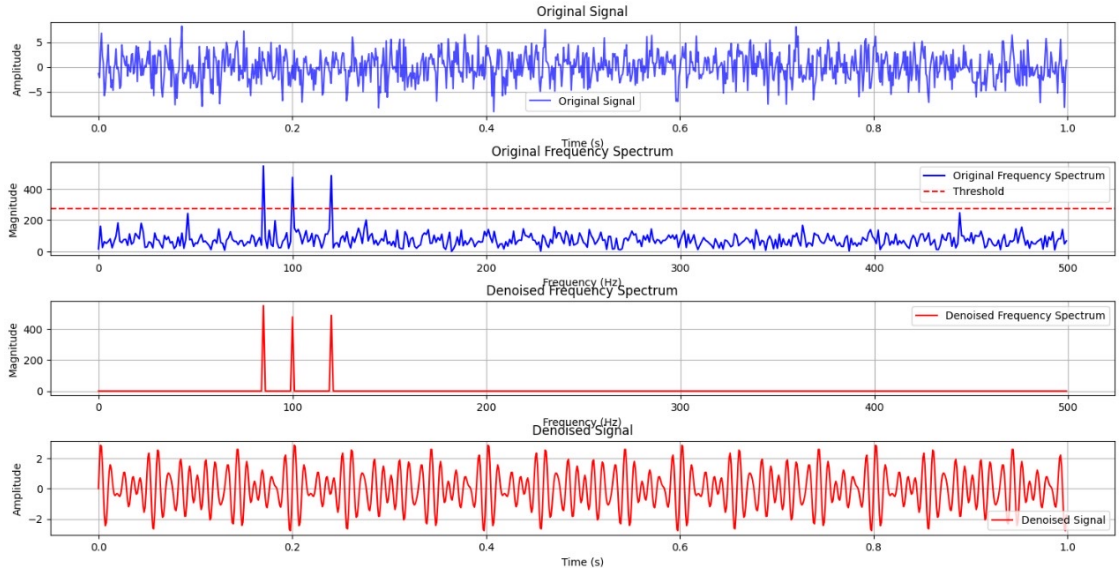
plt.tight_layout()
plt.show()

top_freq_indices = np.argsort(magnitude_clean)[-6:]
print("\nDominant frequencies in the denoised signal:")
for idx in reversed(top_freq_indices):
    if freq[idx] >= 0:
        print(f"Frequency: {freq[idx]:.2f} Hz, Magnitude: {magnitude_clean[idx]:.2f}")
```

Output:

```
Dominant frequencies in the denoised signal:
Frequency: 85.00 Hz, Magnitude: 547.80
Frequency: 120.00 Hz, Magnitude: 485.39
Frequency: 100.00 Hz, Magnitude: 474.50
```

Plot:



**Task 2:**

$$A = \begin{bmatrix} 3 & 2 & 2 \\ 2 & 3 & -1 \end{bmatrix} \qquad A^T A = \begin{bmatrix} 13 & 12 & 4 \\ 12 & 13 & 1 \\ 4 & 1 & 5 \end{bmatrix}$$

$$A A^T = \begin{bmatrix} 17 & 12 \\ 12 & 14 \end{bmatrix}$$

Eigenvalues of $A^T.A \Rightarrow \lambda = 0, \frac{31 \pm 3\sqrt{65}}{2}$

Eigenvalues of $A.A^T \Rightarrow \lambda = \frac{31 \pm 3\sqrt{65}}{2}$

To find eigen vectors:

$$(A^T A - \lambda I)\, x = 0 \quad (x: \text{column vector})$$

$$\Rightarrow V = \begin{bmatrix} -0.707 & 0.190 & -0.681 \\ -0.686 & -0417 & 0.596 \\ -0.170 & 0.889 & 0.426 \end{bmatrix}$$

$$(AA^T - \lambda I)\, x = 0$$

$$\Rightarrow U = \begin{bmatrix} -0.758 & 0.653 \\ -0.653 & -0.758 \end{bmatrix}$$

Singular values: $\sigma_1 = \sqrt{25.61} = 5.06$

$\sigma_2 = \sqrt{5.39} = 2.32 \qquad \sigma_3 = 0$

$$\Rightarrow \Sigma = \begin{bmatrix} 5.06 & 0 & 0 \\ 0 & 2.32 & 0 \end{bmatrix} \Rightarrow A = U \Sigma V^T$$

**Task 3:**

All rank one matrices are not equally valuable. Larger singular values correspond to matrices that contribute more significantly to the image structure.

By setting a threshold on the singular values, we can exclude the matrices that contribute minimally to the image's overall information content.

I think this compression would be less effective on images with random pixel values as they lack inherent structure that can be exploited using SVD.

**Program to compress an image:**

```
import numpy as np
import matplotlib.pyplot as plt
from skimage import color, data, io
from skimage.util import img_as_float

# Load the image with proper escaping for the path
image_path = "image.jpeg"
```

```
image = io.imread(image_path)

# Convert to grayscale (optional, modify if working with color images)
gray_image = color.rgb2gray(image)

# Perform Singular Value Decomposition (SVD)
U, S, Vt = np.linalg.svd(gray_image, full_matrices=False)

# Function to reconstruct image with k singular values
def reconstruct_image(U, S, Vt, k):
    S_reduced = np.diag(S[:k])  # Take the top-k singular values
    U_reduced = U[:, :k]        # Corresponding k columns of U
    Vt_reduced = Vt[:k, :]      # Corresponding k rows of Vt
    return np.dot(U_reduced, np.dot(S_reduced, Vt_reduced))

# Values of k to visualize compression levels
k_values = [5, 20, 50, 100, 200, 500]

# Create subplots
fig, axes = plt.subplots(1, len(k_values) + 1, figsize=(20, 10))

# Display the original image
axes[0].imshow(gray_image, cmap='gray')
axes[0].set_title("Original")
axes[0].axis('off')

# Display reconstructed images with varying k
for i, k in enumerate(k_values):
    compressed_image = reconstruct_image(U, S, Vt, k)
    axes[i + 1].imshow(compressed_image, cmap='gray')
    axes[i + 1].set_title(f"k = {k}")
    axes[i + 1].axis('off')

plt.tight_layout()
plt.show()
```

Output:

